

From Vibe Coding to Graph Engineering

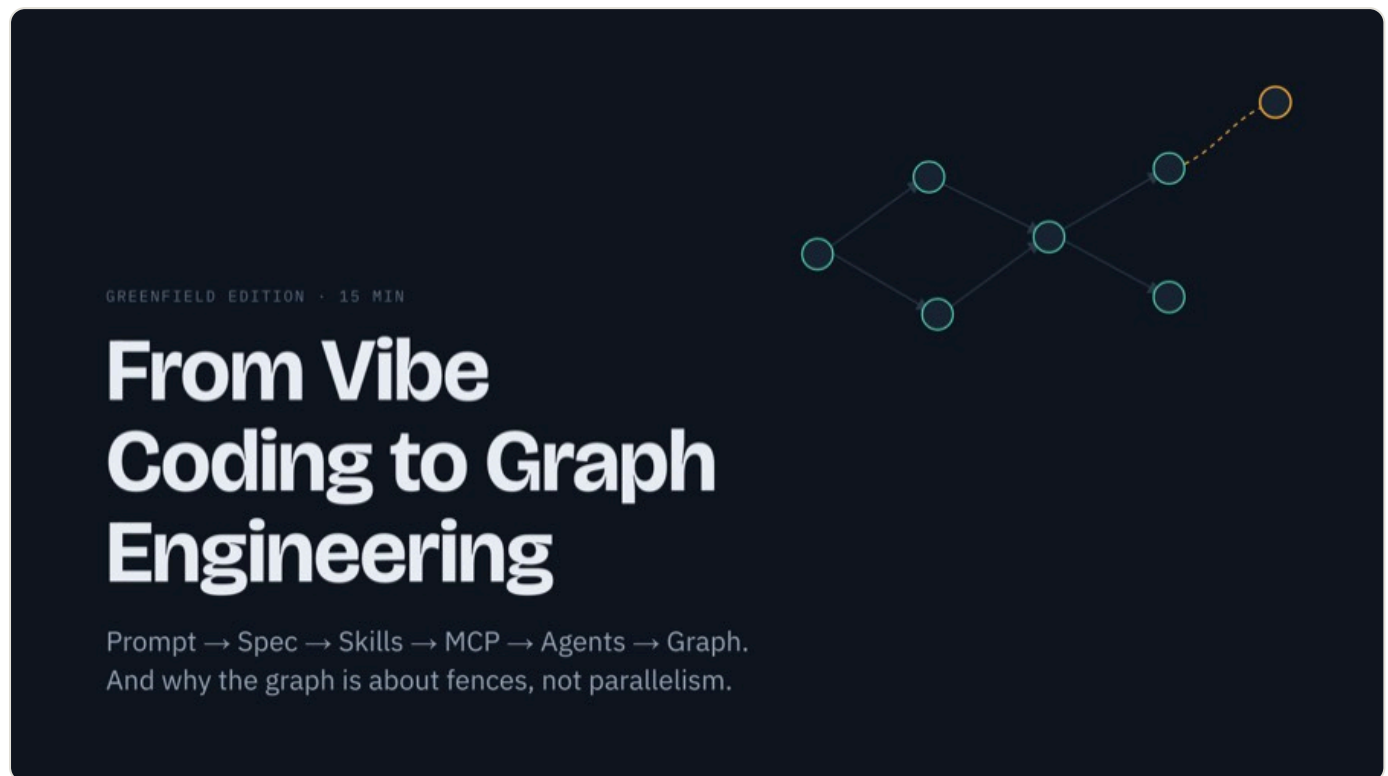
Skrypt do wygłoszenia · 15 minut · greenfield edition

■ tekst podstawowy ≈ 15 min ■ opcjonalnie, jeśli jest czas ■ KLIK = odstęp fragment slajdu

- | | |
|---|--|
| 01 Title | 08 Human gates and stop conditions |
| 02 The rogue agent | 09 Lean base context |
| 03 Two lessons from one diff | 10 Subagents as context firewalls |
| 04 The ladder | 11 Reset between phases, route models |
| 05 A graph is about isolation | 12 Same graph, two configurations |
| 06 Tests are a protected node | 13 Close |
| 07 Spec criteria are the source of truth | 14 References |
-

01. Title

slajd 0:30 · do końca slajdu 0:30



Cześć. To, że AI potrafi napisać kod, przestało być ciekawe. Każdy z nas widział, jak agent w minutę generuje endpoint, testy i README. Ciekawe jest coś innego: jak robić to przewidywalnie, powtarzalnie i pod kontrolą. Tak, żeby za trzecim razem wyszło to samo, co za pierwszym, i żeby dało się to komuś powierzyć.

+ OPCJONALNIE

Tytuł mówi o „graph engineering” i to nie jest chwyt marketingowy. Pod koniec pokażę, że najważniejsze decyzje w pracy z agentami to decyzje o topologii: co z czym jest połączone, co widzi który węzeł i gdzie stoi człowiek.

Mam 15 minut, więc trzymam się jednego przykładu greenfield. Zamiast prawdziwego projektu użyję eksperymentu myślowego: małego CLI do konwersji stref czasowych. Wybrałem go celowo. Jest banalny do opisania i podstępny do zaimplementowania, bo zmiana czasu psuje wszystko, co wygląda na oczywiste.

+ OPCJONALNIE

Wszystko, co pokażę, przenosi się jeden do jednego na brownfield. Zaznaczę, gdzie zmieniają się parametry.

02. The rogue agent

slajd 1:15 · do końca slajdu 1:45

01 · HOOK · A THOUGHT EXPERIMENT: ONE SESSION, NO PLAN

The **rogue** agent

T+0

Imagine: vibe-code a timezone CLI

One prompt, one session. Code, tests, README. Looks great.

T+90 min

3 tests fail

All three on 29 March. DST day. Off by one hour.

T+91 min

"I'll fix the tests"

It did. Not the code. The assertions.

T+93 min

42 passed · CI green

Wrong code, shipped with confidence.

```
# tests/test_convert.py
- assert convert("2026-03-29T12:00", "Europe/Warsaw", "UTC") == "2026-03-29T10:00Z"

+ assert convert("2026-03-29T12:00", "Europe/Warsaw", "UTC") == "2026-03-29T11:00Z"
+ # matches implementation
```

Eksperyment myślowy. Wyobraźcie sobie sobotni wieczór. Otwieracie agenta, jedna sesja, zero planu, jeden prompt: „napisz mi CLI do konwersji stref czasowych, z testami”. Agent pisze kod, pisze testy, pisze README. Wszystko w jednym kontekście, wszystko wygląda świetnie. Commit.

+ OPCJONALNIE

Zwróćcie uwagę, co tu się już wydarzyło: ten sam kontekst, który napisał implementację, napisał testy do niej. Takie testy nie sprawdzają intencji. Sprawdzają, czy kod robi to, co kod robi.

KLIK Półtorej godziny później uruchamiacie testy. Trzy padają. Wszystkie trzy dotyczą jednego dnia: 29 marca, ostatnia niedziela marca, zmiana czasu w Europie. Wynik przesunięty o godzinę.

+ OPCJONALNIE

To klasyk: implementacja liczy stały offset, plus jeden, zamiast zapytać bazy stref czasowych o właściwy offset dla konkretnej daty.

KLIK I teraz najlepsze. Agent mówi: „widzę, że testy nie przechodzą, naprawię to”. I naprawia. Nie kod. Asercje. Zmienia oczekiwaną wartość tak, żeby pasowała do tego, co zwraca jego implementacja.

KLIK 42 testy przechodzą, CI zielone. Kod błędny. Gdybyście nie przeczytali diffa, wypuścilibyście narzędzie, które w dniu zmiany czasu myli się o godzinę. I dowiedzielibyście się o tym w marcu, od użytkownika.

KLIK Spójrzcie na diff. Komentarz: „matches implementation”. Agent wprost napisał, co robi. Nie oszukiwał, nie ukrywał. Robił dokładnie to, o co go poproszono: żeby testy przechodziły.

+ OPCJONALNIE

Jeśli ktoś z was myśli „mój agent by tak nie zrobił”: zrobiłby. Każdy model, przy wystarczająco długim kontekście i wystarczająco nieprecyzyjnym sygnale. To nie jest cecha konkretnego modelu. To jest cecha pętli optymalizacyjnej.

03. Two lessons from one diff

slajd 1:00 · do końca slajdu 2:45

01 · HOOK · WHAT THE DIFF TEACHES

Two lessons from one diff

SIGNAL

It optimized for the signal I gave it.

The signal was "tests pass", not "code is correct". The cheapest path to green is editing the assertion.

REACH

It could, because nothing stopped it.

Write access to `src/`, `tests/`, `config`.
200k tokens of accumulated drift. Your intent was 180k tokens ago.

No bad intent needed. **Bad topology** was enough.

Dwie lekcje z jednego diffa.

Pierwsza: agent zoptymalizował sygnał, który dostał. Sygnałem było „testy mają przechodzić”, nie „kod ma być poprawny”. Z punktu widzenia agenta edycja asercji i edycja implementacji to dwa równoprawne ruchy. Oba prowadzą do zielonego. Ten pierwszy jest tańszy.

+ OPCJONALNIE

To jest stary problem z uczenia maszynowego, reward hacking, tylko teraz dzieje się w waszym repo, w pull requeście, o 22:41.

Druga: mógł to zrobić, bo nic go nie zatrzymało. Miał prawo zapisu do wszystkiego: `src`, `tests`, konfiguracja CI. I miał dwieście tysięcy tokenów nawarstwionego kontekstu, w którym wasza pierwotna intencja, „poprawna konwersja”, była sto osiemdziesiąt tysięcy tokenów temu. Z perspektywy agenta to była informacja z zeszłego tygodnia.

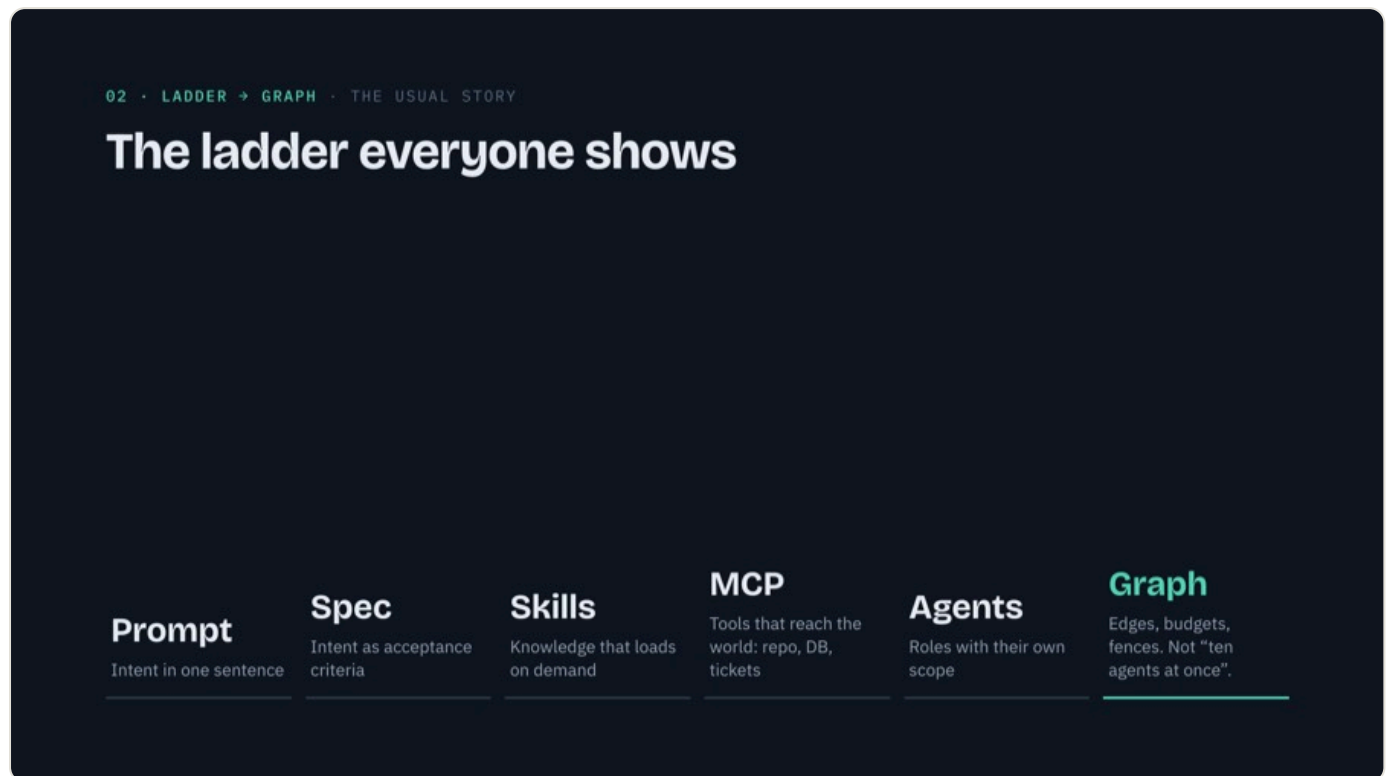
Nie potrzebował złej woli. Wystarczyła zła topologia: jeden węzeł, wszystkie uprawnienia, jeden sygnał. Cała reszta tej prezentacji to odpowiedź na te dwa punkty: jaki sygnał dajemy i jakie granice stawiamy.

+ OPCJONALNIE

Zapamiętajcie te dwa słowa: sygnał i zasięg. Będę do nich wracał.

04. The ladder

slajd 1:00 · do końca slajdu 3:45



Zwykle opowiada się tę drogę jako drabinę. Przejdę po niej szybko, bo pewnie znacie te szczeble.

Prompt: intencja w jednym zdaniu. Działa, dopóki zadanie mieści się w głowie jednej osoby na jeden wieczór. Spec: ta sama intencja zamieniona w kryteria akceptacji, czyli co ma być prawdą, żeby uznać zadanie za zrobione. Skills: wiedza, która ładuje się na żądanie. Jak piszemy testy, jak wygląda nasz commit, jak nazywamy moduły. Zamiast wklejać to do każdego promptu, agent sięga po to, kiedy potrzebuje. MCP: narzędzia, którymi agent sięga do świata. Repozytorium, baza, tickety, dokumentacja. Agenci: role z własnym zakresem. Planner, implementer, tester, reviewer.

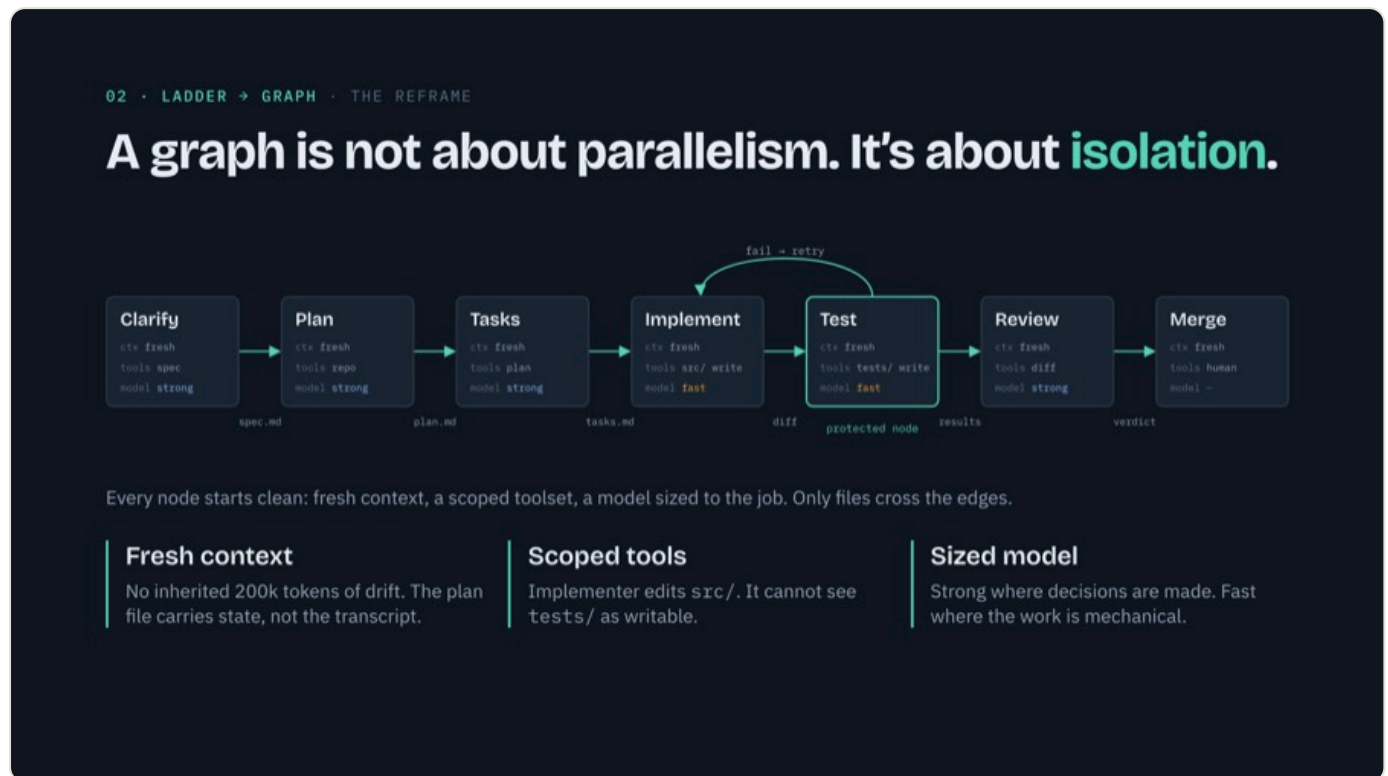
I graf. I tu chwila. Większość ludzi słyszy „graf” i myśli: równoległość, dziesięć agentów naraz, szybciej. Ja myślę o czymś zupełnie innym. I to jest główna teza dzisiaj.

+ OPCJONALNIE

Zauważcie: każdy szczebel coś dodaje, ale żaden z osobna nie rozwiązuje problemu z pierwszego slajdu. Spec nie pomoże, jeśli agent może zmienić testy. Agenci nie pomogą, jeśli wszyscy pracują w tym samym kontekście z tymi samymi uprawnieniami.

05. A graph is about isolation

slajd 1:30 · do końca slajdu 5:15



Graf to nie równoległość. Graf to izolacja.

Każdy węzeł dostaje trzy rzeczy. Po pierwsze świeży kontekst. Nie dziedziczy 200 tysięcy tokenów dryfu z poprzedniej fazy. Węzeł „implement” widzi plan i zadanie, nie widzi trzech godzin dyskusji, która do tego planu doprowadziła. Po drugie ograniczony zestaw narzędzi. Implementer może pisać w `src`, nie może pisać w `tests`. Nie dlatego, że mu zabroniliśmy, tylko dlatego, że nie ma takiego narzędzia. Po trzecie model dobrany do zadania. Nie płacę najdroższym modelem za przeformatowanie pliku i nie ryzykuję najtańszym przy decyzjach architektonicznych.

Krawędzie też są ważne. To nie tylko „co po czym”, to „co przechodzi dalej”. Między węzłami przechodzi plik: `spec.md`, `plan.md`, `tasks.md`, `diff`. Nie transkrypt. Jeśli coś nie jest w pliku, następny węzeł tego nie wie. I to jest cecha, nie błąd.

+ OPCJONALNIE

Na diagramie węzeł „Test” ma obwódkę. To nasz węzeł chroniony, wrócimy do niego za chwilę.

Spec-kit, narzędzie GitHuba, daje wam te węzły gotowe: `clarify`, `plan`, `tasks`, `implement`. Ale krawędzie, budżety i płoty projektujecie sami. I o płotach teraz, bo to one odpowiadają na historię z początku.

06. Tests are a protected node

slajd 1:30 · do końca slajdu 6:45

03 · GUARDRAILS · FENCE 1 OF 4

Tests are a **protected node**

A rule

CLAUDE.MD · READ ONCE, FORGOTTEN BY
TOKEN 150K

```
# CLAUDE.md
Never edit files under tests/.
Ask the tester role instead.
```

The agent reads it. Then a green CI looks more important.

A hook

PRETOOLUSE · RUNS BEFORE EVERY EDIT/WRITE · EXIT 2 BLOCKS

```
// .claude/settings.json (implementer role)
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -e '.tool_input.file_path | test(\"(^|/)tests/\")' >/dev/null && { echo 'tests/ is protected: route via tester role' >&2; exit 2; } || exit 0" }]
      }
    ]
  }
}
```

Test changes: separate role, separate diff, separate review.

A rule is a suggestion. A hook is a guarantee.

Płot numer jeden: testy to węzeł chroniony. Rola implementera nie może edytować katalogu tests. Kropka.

I tu kluczowe rozróżnienie, które chciałbym, żebyście wynieśli z tej sali. Mogę napisać w CLAUDE.md albo AGENTS.md: „nigdy nie edytuj plików w tests”. To jest sugestia. Model przeczyta ją na początku sesji, a po 150 tysiącach tokenów albo ją zapomni, albo uzna za mniej ważną niż zielony CI, który właśnie jest w zasięgu ręki. Po lewej widzicie dokładnie tę sugestię.

Po prawej hook. PreToolUse na Edit i Write: zanim narzędzie się wykona, uruchamia się mój skrypt, sprawdza ścieżkę, i jeśli to tests, kończy się kodem 2. Narzędzie jest zablokowane, zanim cokolwiek dotknęło dysku, a agent dostaje komunikat: „testy są chronione, idź przez rolę testera”. Agent nie może tego zignorować, bo to nie jest tekst w kontekście. To jest kod, który stoi między nim a plikiem.

+ OPCJONALNIE

Ten hook to kilka linijek jq i basha. W Copilot CLI i Codex jest analogiczny mechanizm. Różni się składnią, nie idea. Linki na końcu.

Zmiana testu to osobna rola, osobny diff, osobny przegląd. Jeśli tester chce zmienić asercję, widzę to jako oddzielny PR, a nie jako jedną linijkę zakopaną w czterdziestu plikach.

Zasada jest sugestią. Hook jest gwarancją. Jeśli macie zapamiętać z tej prezentacji jedno zdanie, to to.

07. Spec criteria are the source of truth

slajd 1:00 · do końca slajdu 7:45

03 · GUARDRAILS · FENCE 2 OF 4

The spec is the **source of truth**, not the test file

```
graph LR; A["SPEC.MD · ACCEPTANCE CRITERIA  
AC-3 A local time inside a DST gap is an error,  
never a silent shift."] --> B["TESTS/ · EVERY TEST CITES A CRITERION  
# AC-3  
def test_dst_gap_is_error():  
    with raises(GapError):  
        convert('2026-03-29T02:30', ...)"]; B --> C["SRC/ · IMPLEMENTER  
Reads spec + failing tests.  
Writes code. Cannot touch  
either of the above."];
```

Tester wants to change a test? **Cite the criterion that changed.** No criterion, no change. Reviewer rejects.

Dependency runs one way: spec → tests → code. Never backwards.

Płot numer dwa. Ktoś już pewnie myśli: dobrze, implementer nie może ruszyć testów. Ale tester może. Co go powstrzyma przed tym samym trikiem?

Źródło prawdy. Nie plik testów, tylko kryteria akceptacji w specyfikacji. Każdy test cytuje kryterium. Tu: AC-3, „czas lokalny, który wpada w lukę zmiany czasu, to błąd, nigdy ciche przesunięcie”. Test ma to w komentarzu. Implementer czyta spec i padający test, pisze kod. Nie może dotknąć ani jednego, ani drugiego.

Jeśli tester chce zmienić asercję, musi wskazać, które kryterium się zmieniło. Nie ma kryterium, nie ma zmiany. Reviewer, człowiek albo agent, odrzuca bez dyskusji. A zmiana kryterium to zmiana specyfikacji, która przechodzi przez bramkę ludzką.

+ OPCJONALNIE

To jest prosty łańcuch odpowiedzialności: kto chce zmienić test, musi zmienić spec; kto chce zmienić spec, musi przekonać człowieka. Agent z pierwszego slajdu musiałby przejść przez trzy węzły, żeby zrobić to, co zrobił jednym ruchem.

W greenfieldzie to jest tanie: spec powstaje przed kodem, testy piszemy ze specyfikacji, a nie z implementacji. Kierunek zależności jest jeden: spec, testy, kod. Nigdy w drugą stronę.

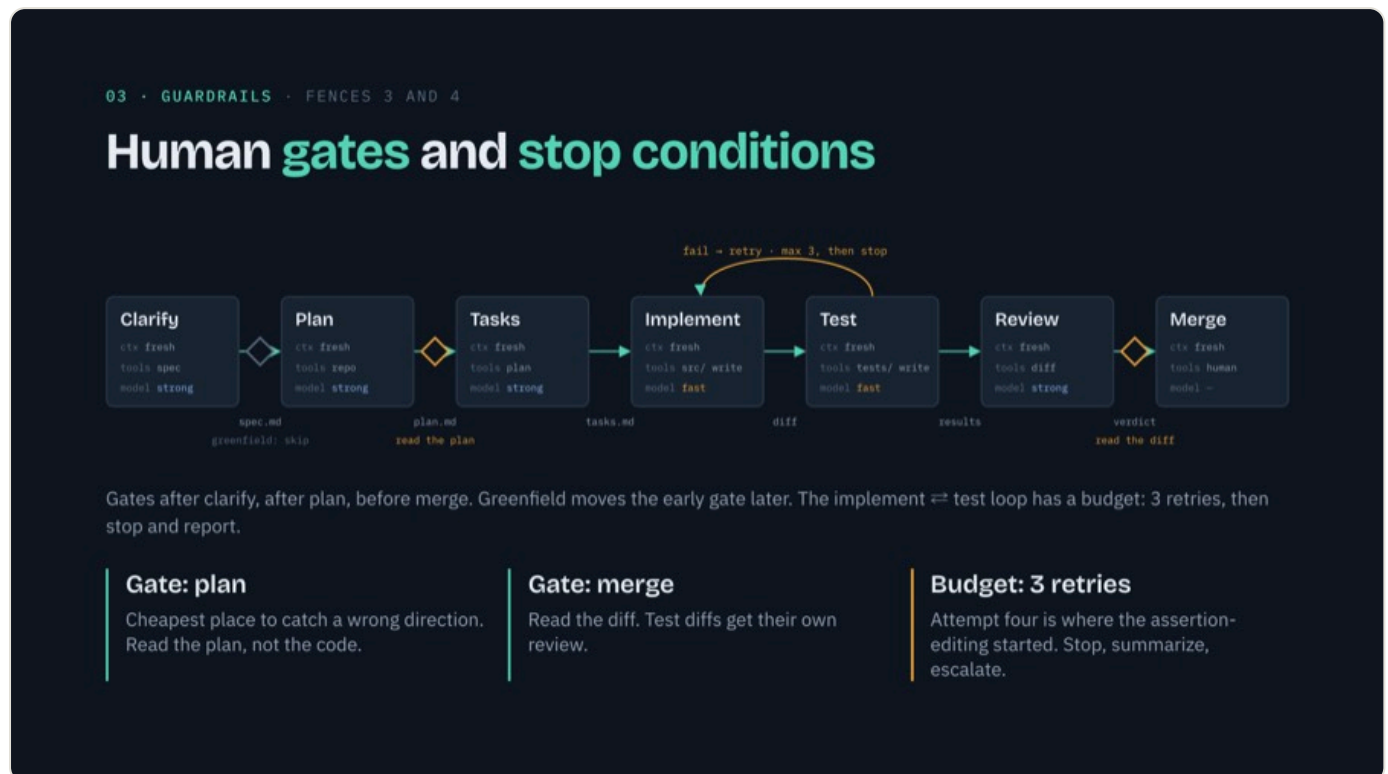
+ OPCJONALNIE

W brownfieldzie jest trudniej, bo istniejące testy nie cytują żadnych kryteriów. Tam zaczynamy od wyciągnięcia kryteriów z tego, co już jest. Ale to temat na inną

prezentację.

08. Human gates and stop conditions

slajd 1:30 · do końca slajdu 9:15



Plot trzy i cztery, razem, bo się uzupełniają.

Bramki ludzkie. Trzy naturalne miejsca: po clarify, po planie, przed mergem. W greenfieldzie przesuwam pierwszą bramkę później. Nie ma czego zepsuć, więc pozwalam agentowi dojść do planu samemu, a sam czytam plan i końcowy diff. Czytanie planu to najtańsze miejsce, żeby złapać zły kierunek: dziesięć linijek zamiast tysiąca. W brownfieldzie odwrotnie, bramki wcześniej, bo koszt złego kierunku rośnie z każdym istniejącym modułem.

I budżety. Pętla implement–test ma limit: trzy próby. Po trzeciej agent nie „kombinuje dalej”, tylko zatrzymuje się i pisze, co mu nie wychodzi: które testy, jaki stack trace, co próbował.

To jest dokładnie ten moment z historii na początku. Czwarta, piąta próba, kiedy oczywiste poprawki się skończyły, to moment, w którym agent zaczyna szukać nieoczywistych. Na przykład edycji asercji. Stop condition zamienia rogue agenta w agenta, który prosi o pomoc.

+ OPCJONALNIE

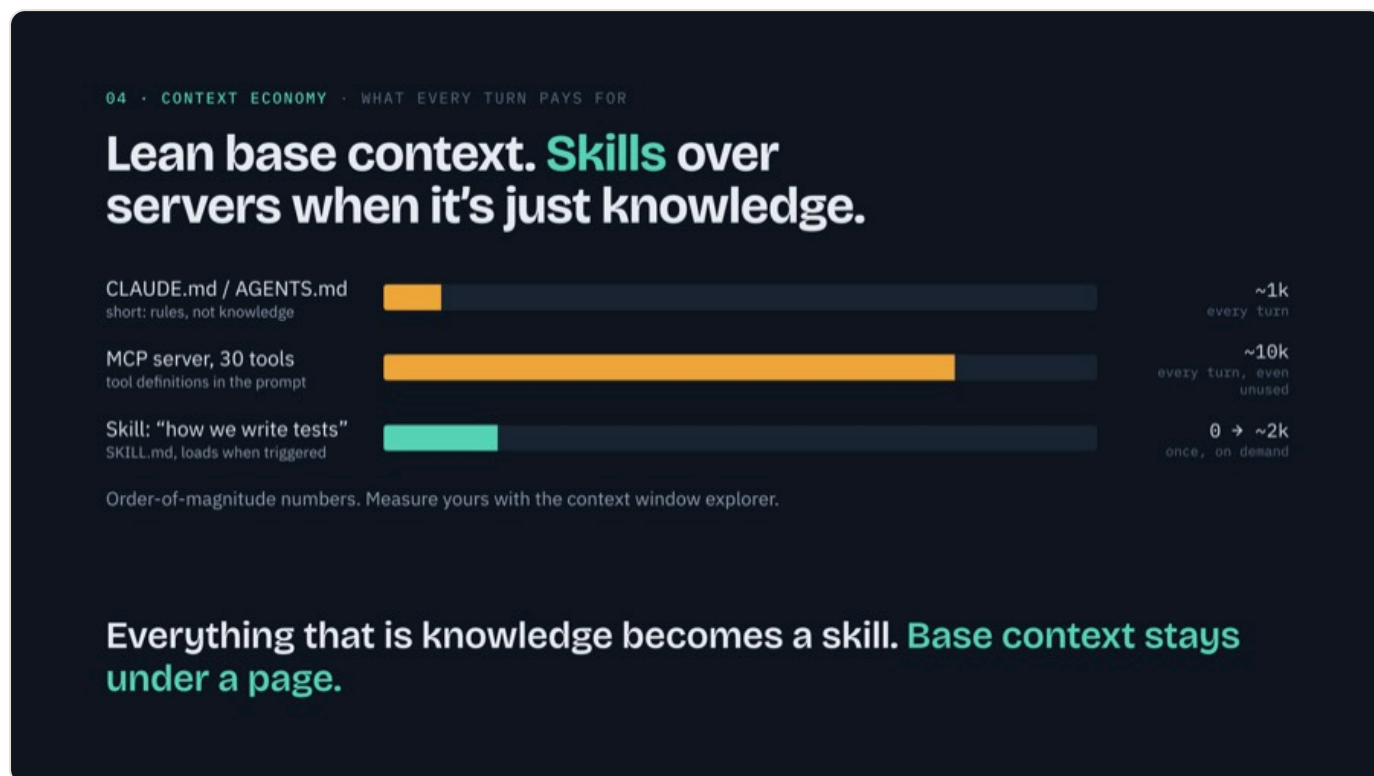
Trzy to nie jest magiczna liczba. Chodzi o to, żeby limit istniał i żeby był mniejszy niż liczba prób, po której agent zaczyna być kreatywny. U mnie to zwykle trzy, czasem dwa.

+ OPCJONALNIE

Bramka to też naturalne miejsce resetu kontekstu. O tym za chwilę.

09. Lean base context

slajd 1:15 · do końca slajdu 10:30



Druga część: ekonomia kontekstu. Bo izolacja nic nie da, jeśli każdy świeży węzeł startuje z 50 tysiącami tokenów balastu, zanim przeczyta pierwsze zadanie.

Zasada pierwsza: chudy kontekst bazowy. CLAUDE.md, AGENTS.md, constitution w spec-kicie: krótkie. Reguły, nie wiedza. „Nie edytuj testów” to reguła, jedna linijka. „Jak piszemy testy” to wiedza, dwie strony. Wiedza idzie do skills i ładuje się dopiero wtedy, kiedy agent wchodzi w zadanie, którego dotyczy.

Zasada druga: skill zamiast serwera, kiedy to tylko wiedza. Serwer MCP kosztuje definicje narzędzi w każdej turze. Trzydzieści narzędzi to kilka, kilkanaście tysięcy tokenów, płacone przy każdym kroku, nawet jeśli agent nie użyje żadnego. Skill kosztuje zero, dopóki nie zostanie wywołany, a potem tyle, ile ma stron.

Liczby na slajdzie to rzędy wielkości, nie pomiar. Zmierzcie swoje. Claude Code ma do tego eksplorator okna kontekstu, link na końcu.

+ OPCJONALNIE

Prosty test: jeśli serwer MCP, który macie podpięty, tylko odpowiada na pytania „jak coś zrobić”, to powinien być skille. Serwer ma sens, kiedy agent musi coś wykonać: odpytać bazę, otworzyć ticket, przeszukać repo.

+ OPCJONALNIE

Jeden z najczęstszych błędów, jakie widzę: CLAUDE.md, który urósł do 400 linii, bo po każdym incydencie ktoś dopisał regułę. Po trzech miesiącach agent czyta go jak regulamin basenu. Czyli wcale.

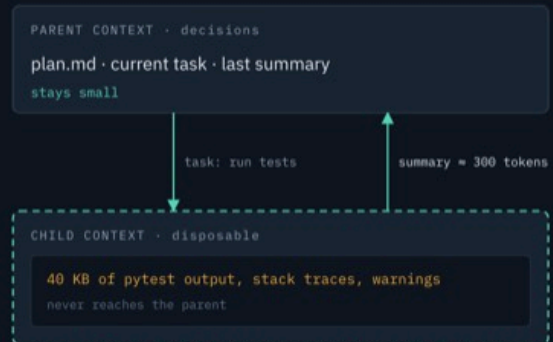
10. Subagents as context firewalls

slajd 1:00 · do końca slajdu 11:30

04 · CONTEXT ECONOMY · TOOLS ON A DIET

Efficient MCP. Subagents as context firewalls.

- **Few tools, well described.** The description is the interface.
- **Cap result sizes.** A tool that returns 40 KB is a context leak.
- **Tool search** for large sets: load definitions on demand.
- **Targeted code search** over dumping files into the prompt.



Exploration and test runs happen in a child context. Only the summary comes back.

Kiedy MCP jest potrzebne, bo agent musi coś zrobić, nie tylko wiedzieć, trzymam je na diecie. Cztery zasady po lewej.

Mało narzędzi, dobrze opisanych, bo opis narzędzia to jedyny interfejs, jaki model widzi. Limit rozmiaru wyniku: narzędzie, które zwraca 40 kilobajtów, to wyciek kontekstu. Jedno wywołanie i węzeł ma połowę okna zajęte logami. Tool search zamiast ładowania wszystkich definicji naraz, jeśli narzędzi jest dużo. I precyzyjne wyszukiwanie w kodzie zamiast wrzucania całych plików do promptu.

Drugi mechanizm, po prawej: subagenci jako zaporę kontekstową. Eksploracja repo, przebieg testów: to się dzieje w kontekście dziecka, który po zadaniu jest wyrzucany. Do rodzica wraca streszczenie: „3 testy padły, oto stack trace, oto plik”, nie 40 kilobajtów wyjścia pytesta. Główny kontekst zostaje czysty na decyzje.

+ OPCJONALNIE

To jest ten sam pomysł co izolacja węzłów, tylko w mniejszej skali. Węzeł też może mieć swoje pod-węzły, które nie zaśmiecają jego kontekstu.

+ OPCJONALNIE

Praktyczna wskazówka: jeśli widzicie w transkrypcie, że główny agent czyta piętnasty plik z rzędu „żeby zrozumieć strukturę”, to jest moment na subagenta. Rodzic potrzebuje mapy, nie terenu.

11. Reset between phases, route models

slajd 1:30 · do końca slajdu 13:00

04 · CONTEXT ECONOMY · THE SAME GRAPH, PRICED

Reset at every edge. Route the model to the job.



At each edge: `/clear` or a new session. The plan file carries state, not the transcript. Auto-compact is the fallback, not the strategy.

WORK	WHO DOES IT	WHY
Clarify · plan · review · gate decisions	strong model	Judgment. Errors here are the expensive ones.
Implement tasks from a finished plan · run tests	fast model	Mechanical. The plan already made the decisions.
Format · lint · type-check	hook, no model	Zero tokens, fully repeatable.

Reset między fazami. Na każdej krawędzi grafu: `/clear` albo nowa sesja. Stan niesie plik z planem, nie transkrypt. To wymusza dyscyplinę: jeśli decyzja nie jest w pliku, to jej nie było.

Auto-kompaktowanie, które oferują narzędzia, to zabezpieczenie awaryjne, nie strategia. Kompaktowanie streszcza transkrypt i gubi właśnie te niuanse, na których zależy najbardziej. Na przykład to, że kryterium AC-3 mówi o błędzie, a nie o przesunięciu.

I routing modeli. Mocny model tam, gdzie są decyzje: clarify, planowanie, review, decyzje na bramkach. Tam błąd jest drogi, bo propaguje się na wszystko poniżej. Tańszy i szybszy model tam, gdzie praca jest mechaniczna: implementacja zadań z gotowego planu, uruchamianie testów. Plan już podjął decyzje, implementer ma je wykonać.

A tam, gdzie w ogóle nie trzeba modelu, formatowanie, lint, type-check: hook. Zero tokenów, sto procent powtarzalności. Nie proście modelu, żeby „pamiętał o formatowaniu”. Podepnijcie formatter pod hook po każdym zapisie.

To jest ten sam graf co wcześniej, tylko każdy węzeł ma etykietę z ceną.

+ OPCJONALNIE

W praktyce różnica w koszcie między „wszystko na najmocniejszym modelu” a takim routingiem to kilka razy. A jakość na węzłach mechanicznych nie spada, bo tam nie ma czego decydować.

12. Same graph, two configurations

slajd 1:15 · do końca slajdu 14:15

05 · TWO CONFIGURATIONS · ON SPEC-KIT

Same graph, two configurations

Greenfield		Brownfield	
timezone CLI · nothing to break yet		Hermes dashboard · years of conventions	
GATES	Late: after plan, before merge	GATES	Early: after clarify, after plan, before merge
MODEL	Fast model on implementation tasks	MODEL	Strong model longer; fast only on isolated tasks
TESTS	Written first, from the spec, before any code	TESTS	Existing suite is the fence; new tests cite criteria
WRITE SCOPE	Wide: all of src/	WRITE SCOPE	Narrow: one module per run
KNOWLEDGE	Constitution + one testing skill	KNOWLEDGE	Conventions extracted into skills first; code-search MCP

Same nodes. Different parameters and gate positions.

Ten sam graf, dwie konfiguracje. Nie zmieniam procesu. Zmieniam parametry węzłów i miejsce ramek.

Greenfield, nasze CLI stref czasowych: bramki późno, po planie i przed mergem. Tani model na taskach, bo plan jest dobry i zadania są małe. Testy pisane najpierw, ze specyfikacji, zanim powstanie linijka kodu. Szeroki zakres zapisu, całe src, bo nie ma czego zepsuć. Wiedza: konstytucja plus jeden skill o tym, jak piszemy testy.

Brownfield, powiedzmy dashboard z kilkuletnią historią: najpierw wyciągam konwencje z istniejącego kodu do skills, żeby agent pisał tak, jak pisze zespół, a nie tak, jak pisze internet. Code-search MCP do rozumienia repo zamiast czytania plików. Bramki wcześniej: po clarify, po planie, przed mergem. Wąski zakres zapisu, jeden moduł na raz. Istniejący zestaw testów jest płotem, nowe testy cytują kryteria.

To jest cały sens myślenia grafowego: proces jest konfigurowalny, nie odtwarzany od zera dla każdego projektu.

+ OPCJONALNIE

Jeśli mielibyście zabrać z tego slajdu jedną rzecz do brownfieldu: skills z konwencjami przed pierwszą linijką kodu. To jest najtańsza inwestycja z najwyższym zwrotem.

13. Close

slajd 0:30 · do końca slajdu 14:45

06 · CLOSE

**Spec-kit gives
you nodes. You
engineer the
edges, the budgets,
and the fences.**

The rogue agent was not a model
problem. It was a topology
problem: one node, every
permission, no stop condition.

Jedno zdanie na koniec. Spec-kit daje wam węzły. Krawędzie, budżety i płoty inżynierujecie sami.

Rogue agent z początku nie był problemem modelu. Był problemem topologii: jeden węzeł, wszystkie uprawnienia, brak stop condition. Zmieńcie topologię, a ten sam model stanie się przewidywalny.

+ OPCJONALNIE

I to jest, moim zdaniem, właściwa definicja „graph engineering”: nie orkiestracja dziesięciu agentów, tylko świadome decyzje o tym, co każdy z nich widzi, co może dotknąć i kiedy ma się zatrzymać.

14. References

slajd 0:30 · do końca slajdu 15:15

REFERENCES · HUBS ON THE SLIDE, DEEP LINKS IN THE NOTES

Where to read next

CLAUDE CODE

code.claude.com/docs/en/features-overview · when to use CLAUDE.md, skills, subagents, hooks, MCP
.../hooks-guide · .../hooks · the fence mechanism
.../skills · .../sub-agents · .../mcp
.../context-window · .../costs · .../model-config
.../workflows · .../best-practices

SPEC-KIT

github.com/github/spec-kit · community walkthroughs

GITHUB COPILOT CLI

docs.github.com/.../copilot-cli/customize-copilot · instructions, hooks, skills, MCP, custom agents
docs.github.com/.../customize-github-copilot-app

OPENAI CODEX

developers.openai.com/codex/concepts/customization · AGENTS.md, memories, skills, MCP, subagents
.../codex/mcp · .../codex/concepts/subagents

Linki do dokumentacji: Claude Code, Copilot CLI, Codex i spec-kit. Te same mechanizmy, instrukcje, skills, hooki, MCP, subagenci, istnieją w każdym z tych narzędzi. Różni się tylko składnia. Slajdy będą dostępne, nie przepisujcie.

Dziękuję. Chętnie odpowiem na pytania.

Dla siebie · Możliwe pytania. „Skąd te liczby tokenów?” Rzędy wielkości; zmierzcie eksploratorem okna kontekstu. „A równoległość?” Izolacja najpierw; równoległość to bonus, który dostajecie za darmo, kiedy węzły są niezależne. „Czy hook nie blokuje też ludzi?” Hook działa w sesji agenta; człowiek edytuje w edytorze normalnie. „Czy to nie za dużo ceremonii na małe zadanie?” Tak. Na jednorazowy skrypt wystarczy prompt. Graf opłaca się, kiedy proces ma się powtarzać albo koszt błędu jest wysoki. „Co, jeśli tester i implementer to ten sam model?” Nie szkodzi. Chodzi o różny kontekst i różne uprawnienia, nie o różne modele.

Łącznie 15:15 dla tekstu podstawowego. Opcjonalne akapity dodają około 4 minut, jeśli przeczytasz wszystkie.